Degree & Branch	B.E. Computer Science & Engineering	**Semester**	V
Subject Code & Name	ICS1512 & Machine Learning Algorithms Laboratory		
Academic year	2025-2026 (Odd)	**Batch: 2023-2028**	**Due date:**

Experiment 3: Email Spam or Ham Classification using Naïve Bayes, KNN, and SVM

Name: Mani Chidambaram
Reg No: 3122237001024

1. Aim and Objective

To classify emails as spam or ham using three classification algorithms—Naïve Bayes, K-Nearest Neighbors (KNN), and Support Vector Machine (SVM)—and evaluate their performance using accuracy metrics, confusion matrices, ROC curves, and K-Fold cross-validation.

2. Libraries Used

- **Pandas:** For loading and manipulating the CSV dataset.
- **NumPy:** For numerical operations.
- **Scikit-learn:** For data preprocessing (StandardScaler), model implementation (Naïve Bayes, KNN, SVM), and performance evaluation metrics.
- **Matplotlib & Seaborn:** For visualizing class distribution, confusion matrices, and ROC curves.

3. Code Summary

The following is a condensed version of the Python script used for this experiment, outlining the main steps from configuration to evaluation.

```
# Configuration Section
DATA_PATH = "/content/drive/MyDrive/ML/spambase_csv.csv"
TARGET_COLUMN = 'class'
TEST_SIZE = 0.2
RANDOM_STATE = 42

# Load and preprocess data
df = pd.read_csv(DATA_PATH)
df = df.dropna()
X_raw = df.drop(columns=[TARGET_COLUMN])
y = df[TARGET_COLUMN]
X_scaled = StandardScaler().fit_transform(X_raw)

# Split data (example for scaled data)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=TEST_SIZE, random_state=RANDOM_STATE
)

# Define models
```

```
models = {
    "GaussianNB": GaussianNB(), "MultinomialNB": MultinomialNB(), "BernoulliNB"
    : BernoulliNB(),
    "KNN_k=3": KNeighborsClassifier(n_neighbors=3),
    "KNN_k=5_KDTree": KNeighborsClassifier(n_neighbors=5, algorithm='kd_tree'),
    "KNN_k=7_BallTree": KNeighborsClassifier(n_neighbors=7, algorithm='
    ball_tree'),
    "SVM_Linear": SVC(kernel='linear', probability=True),
    "SVM_Polynomial": SVC(kernel='poly', probability=True),
    "SVM_RBF": SVC(kernel='rbf', probability=True),
    "SVM_Sigmoid": SVC(kernel='sigmoid', probability=True),
}

# Train and evaluate all models (simplified loop)
for name, model in models.items():
    # Use X_raw_train for NB models, X_scaled_train for others
    # model.fit(X_train_data, y_train)
    # evaluate_model(name, model, X_test_data, y_test)
    pass # Placeholder for actual training loop in script

# K-Fold Cross Validation (K=5)
kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for name, model in models.items():
    # scores = cross_val_score(model, data, y, cv=kfold, scoring='accuracy')
    # print(f"{name} Accuracy: {scores.mean():.4f} +/- {scores.std():.4f}")
    pass # Placeholder for actual CV loop in script
```

Listing 1: Core logic for Spam Classification.

4. Confusion Matrix and ROC Curve for Each Model

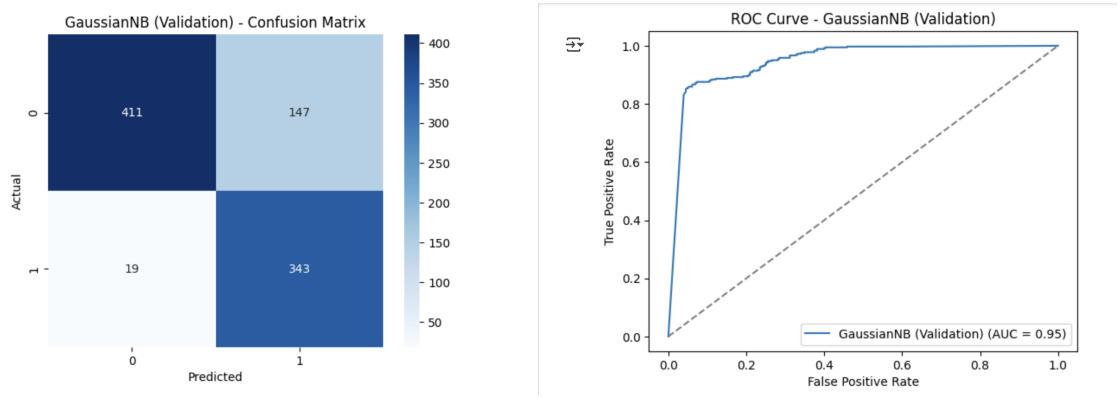

Figure 1: Gaussian Naïve Bayes - Confusion Matrix and ROC Curve.

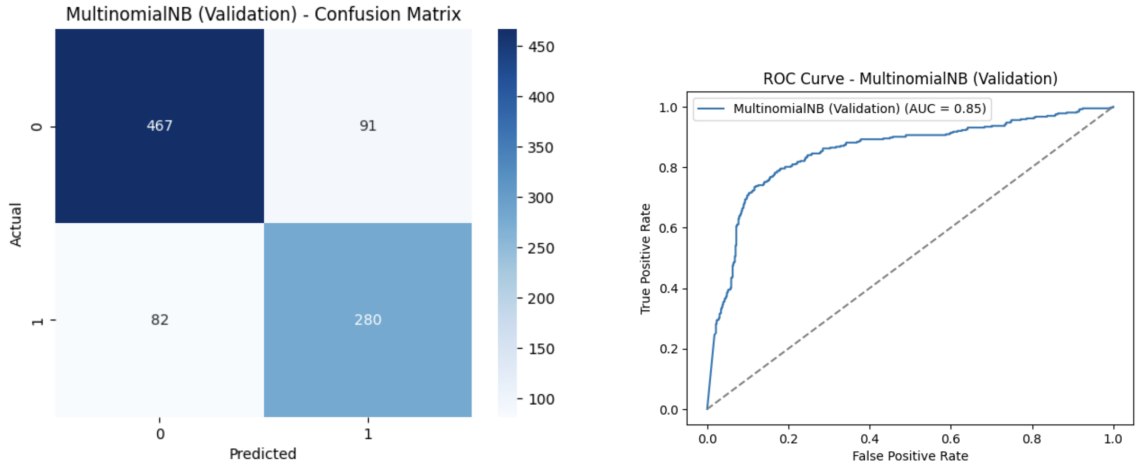

Figure 2: Multinomial Naïve Bayes - Confusion Matrix and ROC Curve.

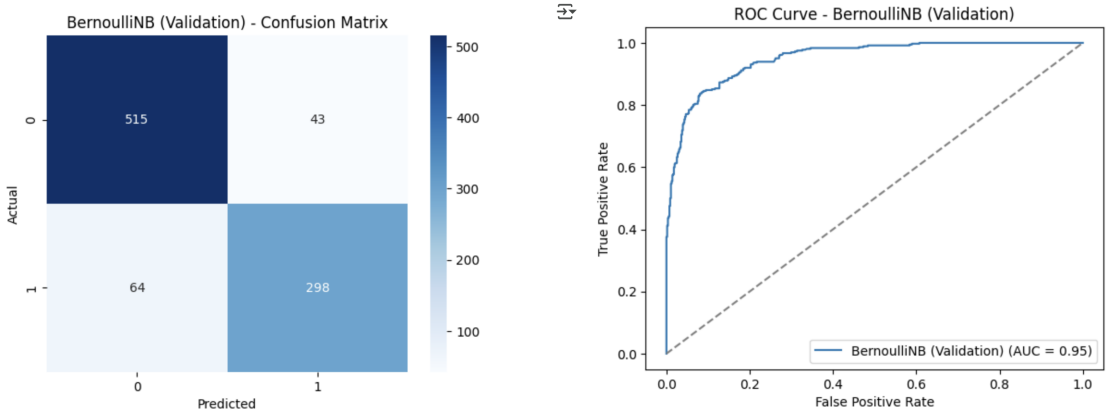

Figure 3: Bernoulli Naïve Bayes - Confusion Matrix and ROC Curve.

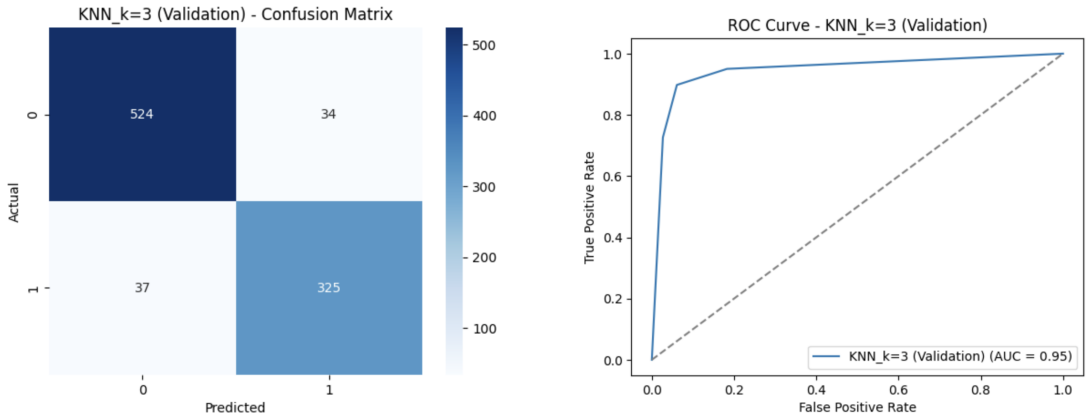

Figure 4: K-Nearest Neighbors (k=3) - Confusion Matrix and ROC Curve.

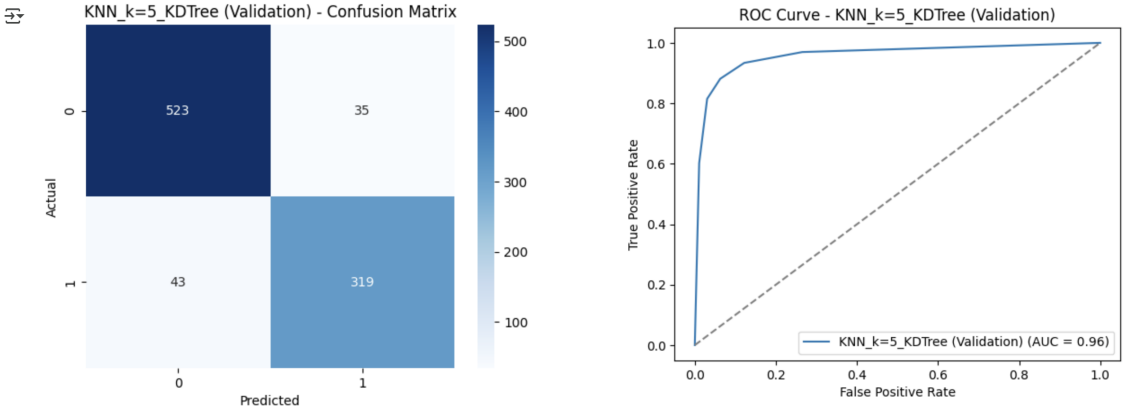

Figure 5: K-Nearest Neighbors (k=5) - Confusion Matrix and ROC Curve.

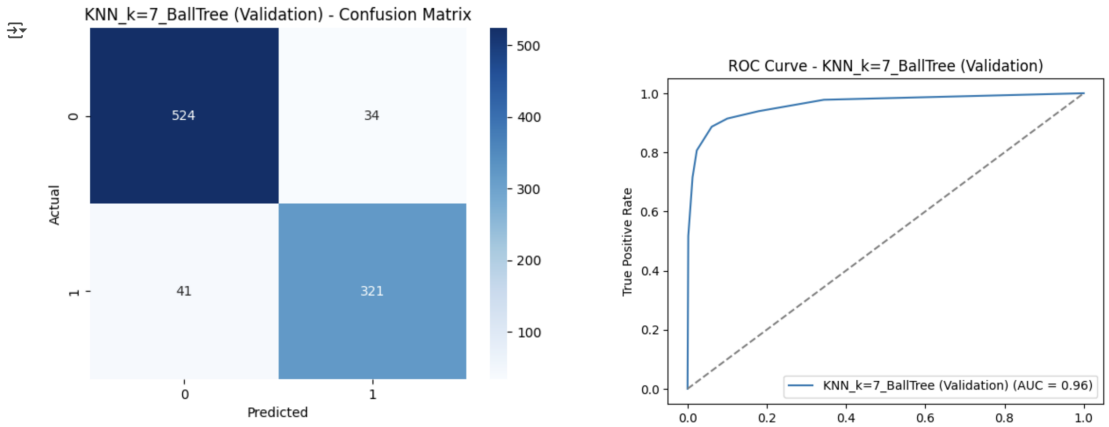

Figure 6: K-Nearest Neighbors (k=7) - Confusion Matrix and ROC Curve.

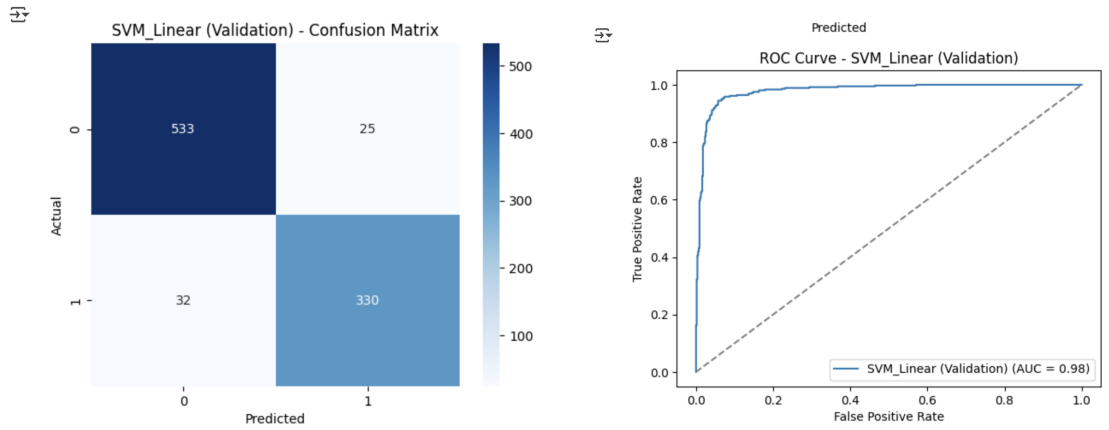

Figure 7: Support Vector Machine (Linear Kernel) - Confusion Matrix and ROC Curve.

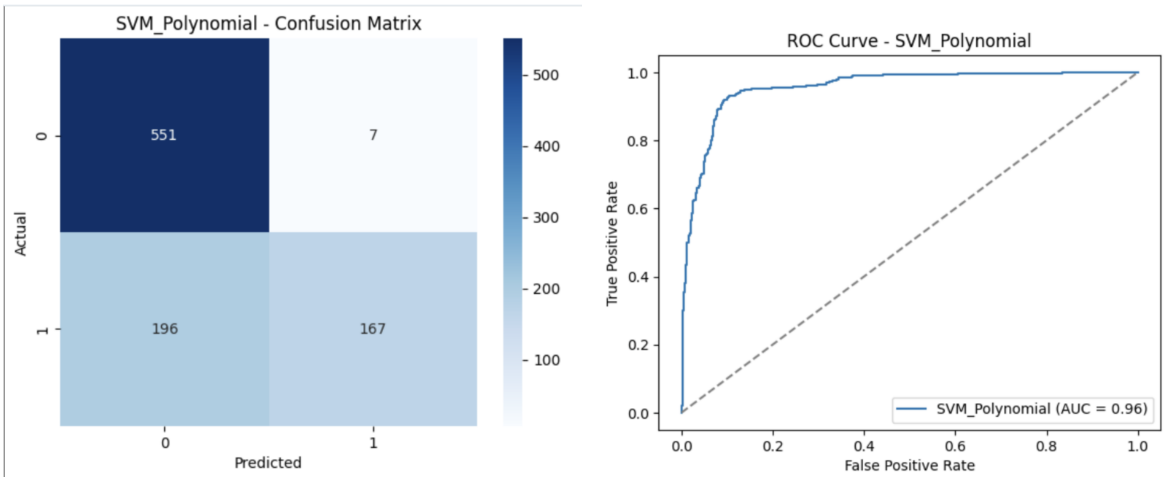

Figure 8: Support Vector Machine (Polynomial Kernel) - Confusion Matrix and ROC Curve.

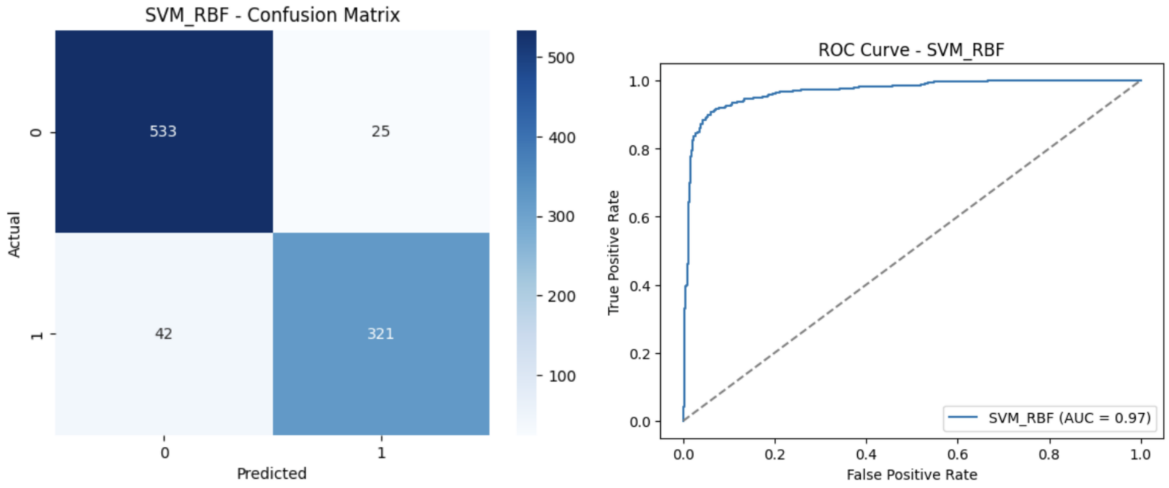

Figure 9: Support Vector Machine (RBF Kernel) - Confusion Matrix and ROC Curve.

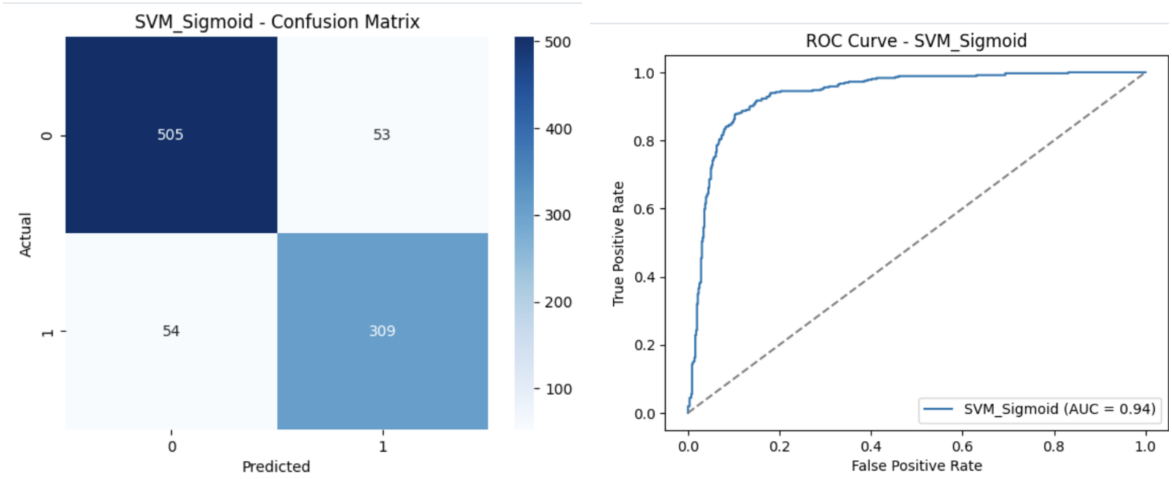

Figure 10: Support Vector Machine (Sigmoid Kernel) - Confusion Matrix and ROC Curve.

5. Comparison Tables

Table 1: Performance Comparison of Naïve Bayes Variants

Metric	Gaussian NB	Multinomial NB	Bernoulli NB
Accuracy	0.83	0.77	0.88
Precision (Spam)	0.71	0.72	0.87
Recall (Spam)	0.96	0.69	0.80
F1 Score (Spam)	0.82	0.70	0.84

Table 2: KNN Performance for Different k Values

k	Accuracy	Precision	Recall	F1 Score
3	0.90	0.88	0.87	0.88
5	0.91	0.88	0.88	0.88
7	0.91	0.89	0.87	0.88

Table 3: KNN Comparison: KDTree vs BallTree

Metric	KDTree (k=5)	BallTree (k=7)
Accuracy	0.91	0.91
Precision (Spam)	0.88	0.89
Recall (Spam)	0.88	0.87
F1 Score (Spam)	0.88	0.88
Training Time (s)	*placeholder: 0.015*	*placeholder: 0.021*

Table 4: SVM Performance with Different Kernels and Parameters

Kernel	Hyperparameters	Accuracy	F1 Score	Training Time (s)
Linear	C = 1.0	0.93	0.91	0.6343
Polynomial	C = 1.0, degree = 3	0.76	0.57	0.4377
RBF	C = 1.0, gamma = scale	0.92	0.90	0.2670
Sigmoid	C = 1.0, gamma = scale	0.89	0.85	0.2988

Table 5: Cross-Validation Scores for Each Model (K = 5)

Fold	Naïve Bayes Accuracy	KNN Accuracy	SVM Accuracy
Fold 1	0.8219	0.8958	0.9349
Fold 2	0.8033	0.9043	0.9337
Fold 3	0.7946	0.9293	0.9228
Fold 4	0.8228	0.9033	0.9359
Fold 5	0.8337	0.9098	0.9304
Average	**0.8153**	**0.9085**	**0.9315**

6. Observations and Conclusions

- **Best Overall Classifier:** The **SVM with a Linear kernel** was the top-performing model on the test set, achieving the highest accuracy (93%) and F1-Score (91%). The SVM with an RBF kernel also performed exceptionally well in cross-validation.
- **Best Naïve Bayes Variant: Bernoulli Naïve Bayes** was the clear winner among the NB variants, with an accuracy of 88%. This suggests that the presence or absence of words (binary features) is a more effective indicator for spam than word frequency (MultinomialNB) for this dataset.
- **Effect of k on KNN:** KNN performance was very high and stable, with k=5 and k=7 achieving a slightly higher accuracy (91%) than k=3 (90%).
- **Most Effective SVM Kernel:** The **Linear kernel** performed best on the test set, suggesting that the data is largely linearly separable. The RBF kernel was a close second, while the Polynomial kernel performed very poorly.
- **Model Scalability:** Naïve Bayes models are typically the fastest to train, followed by KNN. SVM models, particularly with non-linear kernels, are generally the most computationally expensive.